

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected simple graph (no self-loops) represented as an adjacency matrix, where a value of 1 indicates an edge exists between two vertices and a value of 0 indicates no edge. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle starting and ending at vertex 0. If no cycle is found, the program should output "No solution".

Your task is to help Bob implement this.

Input Format

The first line contains an integer V, representing the number of vertices.

The following V lines each contain V space-separated integers (0 or 1), representing the adjacency matrix. The matrix is symmetric (undirected graph) and has zeros on the diagonal (no self-loops)

Output Format

If a Hamiltonian cycle is found, the first line prints "Solution found" and the second line prints the cycle as a series of space-separated integers, representing the vertices starting and ending at vertex 0.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

0 1 0 1 1 0 0 0

1 0 1 0 0 1 0 0

0 1 0 1 0 0 1 0

1 0 1 0 0 0 0 1

1 0 0 0 0 1 0 1

0 1 0 0 1 0 1 0

0 0 1 0 0 1 0 1

0 0 0 1 1 0 1 0

Output: Solution found

0 1 2 3 7 6 5 4 0

Answer

```
#include <stdio.h>
```

```
#define MAX 8
```

```
int graph[MAX][MAX];
```

```
int path[MAX];
```

```
int visited[MAX];
```

```
int V;
```

```
int isSafe(int v, int pos) {
```

```
    if (graph[path[pos - 1]][v] == 0)
```

```

        return 0;
    if (visited[v])
        return 0;

    return 1;
}

int solve(int pos) {
    if (pos == V) {
        if (graph[path[pos - 1]][path[0]] == 1)
            return 1;
        else
            return 0;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;
            visited[v] = 1;

            if (solve(pos + 1))
                return 1;

            visited[v] = 0;
        }
    }

    return 0;
}

```

```

int main() {
    scanf("%d", &V);

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    for (int i = 0; i < V; i++)
        visited[i] = 0;
}

```

```
path[0] = 0;
visited[0] = 1;
```

```
if (solve(1)) {
    printf("Solution found\n");
    for (int i = 0; i < V; i++) {
        printf("%d ", path[i]);
    }
    printf("0");
} else {
    printf("No solution");
}
```

```
return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected complete graph with N vertices ($N \geq 3$), find the number of distinct Hamiltonian cycles in the graph using backtracking. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex.

Two Hamiltonian cycles are considered identical if one can be obtained from the other by rotation (changing the starting vertex) or reversal (traversing in the opposite direction).

Example:

For $N=4$, the vertices are 0, 1, 2, 3. The 3 distinct Hamiltonian cycles are:

0 1 2 3 00 1 3 2 00 2 1 3 0

Output: 3

Input Format

The input consists of a single integer N , representing the number of vertices in the complete graph.

Output Format

Print a single integer on its own line representing the number of distinct Hamiltonian cycles in the complete graph with N vertices.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 60

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int n;  
int visited[MAX];  
int path[MAX];  
long long count = 0;
```

```
void dfs(int pos) {  
    if (pos == n) {  
        // valid cycle found (last connects to 0 automatically in complete graph)  
        count++;  
        return;  
    }
```

```
    for (int i = 1; i < n; i++) {  
        if (!visited[i]) {  
            visited[i] = 1;  
            path[pos] = i;  
  
            dfs(pos + 1);  
  
            visited[i] = 0;
```

```
        }  
    }  
}
```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    path[0] = 0;

    dfs(1);

    // divide by 2 for reverse duplicates
    printf("%lld", count / 2);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

A college is organizing an event and has collected donations from N sponsors. Each sponsor contributes a fixed amount. The event coordinator wants to know all possible groups of sponsors whose donation sum matches exactly the required event budget S.

Write a program using Backtracking to find all subsets of sponsor contributions whose sum is exactly equal to S.

A subset may contain any number of sponsors, but each sponsor amount can be used only once.

Formula:

A subset is valid if: $a_1 + a_2 + \dots + a_k = S$

Input Format

The first line contains an integer N, representing the number of sponsors.

The second line contains N space-separated integers in ascending order,

representing the donation amounts.

The third line contains an integer S, representing the required budget.

Output Format

Print all subsets whose sum is exactly S, one subset per line. Within each subset, print the elements in the order they appear in the input, each followed by a space. Subsets are printed in the order they are discovered during backtracking (subsets starting with smaller elements appear first).

If no subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
10 20 30 40
50
Output: 10 40
20 30

Answer

```
#include <stdio.h>
```

```
int N, S;  
int arr[20];  
int path[20];  
int found = 0;
```

```
void printSubset(int len) {  
    for (int i = 0; i < len; i++) {  
        printf("%d ", path[i]);  
    }  
    printf("\n");  
}
```

```

void solve(int idx, int sum, int len) {
    if (sum == S) {
        found = 1;
        printSubset(len);
        return;
    }

    if (idx == N || sum > S)
        return;

    // Include current element
    path[len] = arr[idx];
    solve(idx + 1, sum + arr[idx], len + 1);

    // Exclude current element
    solve(idx + 1, sum, len);
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &S);

    solve(0, 0, 0);

    if (!found) {
        printf("No subset found");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Emily is working on arranging queens on a chessboard where one queen

must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

A queen can attack any cell in its row, column, or diagonal. Row and column indices are 0-based.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

The first line of output consists of a single floating-point number representing the maximum total value, formatted to two decimal places.

The next n lines each consist of a single floating-point number representing the fraction x_i of each item selected, formatted to two decimal places. x_i represents the fraction of the i-th item taken (1.00 means the item is fully taken, a value between 0.00 and 1.00 means a fraction is taken). The fractions should be printed in the same order as the input items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0

1

Output: . Q . .

. . . Q

Q . .
. Q .

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int N;  
int board[MAX][MAX];  
int col[MAX];  
int diag1[MAX * 2];  
int diag2[MAX * 2];  
int found = 0;
```

```
void printBoard() {  
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++) {  
            if (board[i][j])  
                printf("Q ");  
            else  
                printf(". ");  
        }  
        printf("\n");  
    }  
    printf("\n");  
}
```

```
void solve(int row) {  
    if (row == N) {  
        found = 1;  
        printBoard();  
        return;  
    }  
}
```

```
// Skip fixed queen row  
int fixedRow = -1, fixedCol = -1;
```

```
// find fixed queen position  
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {
```

```
        if (board[i][j] == 2) { // fixed queen marker
            fixedRow = i;
            fixedCol = j;
        }
    }
}
```

```
if (row == fixedRow) {
    solve(row + 1);
    return;
}
```

```
for (int c = 0; c < N; c++) {
    if (col[c] || diag1[row - c + N] || diag2[row + c])
        continue;

    board[row][c] = 1;
    col[c] = diag1[row - c + N] = diag2[row + c] = 1;

    solve(row + 1);

    board[row][c] = 0;
    col[c] = diag1[row - c + N] = diag2[row + c] = 0;
}
}
```

```
int main() {
    int i, j;
    scanf("%d", &N);
    scanf("%d", &i);
    scanf("%d", &j);

    // initialize
    for (int a = 0; a < N; a++)
        for (int b = 0; b < N; b++)
            board[a][b] = 0;

    board[i][j] = 2; // fixed queen marker

    col[j] = 1;
    diag1[i - j + N] = 1;
```

```
    diag2[i + j] = 1;  
    solve(0);  
    if (!found)  
        printf("No solution");  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an adjacency matrix $adj[][]$ of an undirected graph consisting of N vertices, find a Hamiltonian Path in the graph if one exists. If found, print the path as a sequence of vertex indices separated by $\rightarrow$. Otherwise, print No Hamiltonian Path found.

A Hamiltonian path visits each and every vertex of the graph exactly once. The adjacency matrix is symmetric with zeros on the diagonal (no self-loops).

Write a program using Backtracking to find the Hamiltonian Path.

Examples:

Input: $adj[][] = \{\{0, 1, 1, 1, 0\}, \{1, 0, 1, 0, 1\}, \{1, 1, 0, 1, 1\}, \{1, 0, 1, 0, 0\}\}$

Output: Yes

Explanation:

There exists a Hamiltonian Path for the given graph, as shown in the image below:

Input: $\text{adj}[][] = \{\{0, 1, 0, 0\}, \{1, 0, 1, 1\}, \{0, 1, 0, 0\}, \{0, 1, 0, 0\}\}$

Output: No

Input Format

The first line contains an integer N, representing the number of vertices in the graph.

The next N lines each contain N space-separated integers (0 or 1), representing the adjacency matrix of the graph.

Output Format

If a Hamiltonian path exists, print the vertices of the path separated by -> on a single line.

If no Hamiltonian path exists, print: No Hamiltonian Path found

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 1 1 1 0

1 0 1 0 1

1 1 0 1 1

1 0 1 0 0

0 1 1 0 0

Output: 0 -> 1 -> 4 -> 2 -> 3

Answer

```
#include <stdio.h>
```

```
int N;  
int adj[10][10];  
int path[10];  
int visited[10];
```

```
int found = 0;
```

```
int isSafe(int v, int pos) {  
    if (adj[path[pos - 1]][v] == 0)  
        return 0;  
    if (visited[v])  
        return 0;  
    return 1;  
}
```

```
void hamPath(int pos) {  
    if (found) return;
```

```
    if (pos == N) {  
        found = 1;  
        for (int i = 0; i < N; i++) {  
            printf("%d", path[i]);  
            if (i != N - 1) printf(" -> ");  
        }  
        return;  
    }
```

```
    for (int v = 0; v < N; v++) {  
        if (isSafe(v, pos)) {  
            path[pos] = v;  
            visited[v] = 1;
```

```
            hamPath(pos + 1);
```

```
            visited[v] = 0;
```

```
            if (found) return;
```

```
        }
```

```
    }
```

```
}
```

```

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    for (int start = 0; start < N; start++) {
        for (int i = 0; i < N; i++) visited[i] = 0;

        path[0] = start;
        visited[start] = 1;

        hamPath(1);

        if (found) break;
    }

    if (!found) {
        printf("No Hamiltonian Path found");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

You are given an undirected graph with N vertices. Implement a function `longest_hamiltonian_cycle` to find the longest Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same.

Input Format

The first line of input consists of an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at graph[i][j] is 1 if there is an edge between vertex i and vertex j, and 0 otherwise.

Output Format

If a Hamiltonian cycle exists, print "Longest Hamiltonian Cycle:" followed by the vertices of the longest cycle found in order within square bracket separated by a comma.

If no Hamiltonian cycle exists, print "No Hamiltonian Cycle found".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

1 0 1 1

1 1 0 1

0 1 1 0

Output: Longest Hamiltonian Cycle: [0, 1, 3, 2]

Answer

```
#include <stdio.h>
```

```
int N;
```

```
int graph[10][10];
```

```
int path[10], bestPath[10];
```

```
int visited[10];
```

```
int bestLen = 0;
```

```
int isSafe(int v, int pos) {  
    if (!graph[path[pos - 1]][v]) return 0;  
    if (visited[v]) return 0;  
    return 1;  
}
```

```
void copyPath(int len) {
```

```
    for (int i = 0; i < len; i++) {  
        bestPath[i] = path[i];  
    }  
}
```

```
void backtrack(int pos, int start) {  
    if (pos == N) {  
        if (graph[path[pos - 1]][start]) {  
            if (N > bestLen) {  
                bestLen = N;  
                copyPath(N);  
            }  
        }  
    }  
    return;  
}
```

```
    for (int v = 0; v < N; v++) {  
        if (isSafe(v, pos)) {  
            path[pos] = v;  
            visited[v] = 1;  
  
            backtrack(pos + 1, start);  
  
            visited[v] = 0;  
        }  
    }  
}
```

```
int main() {  
    scanf("%d", &N);
```

```
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++) {  
            scanf("%d", &graph[i][j]);  
        }  
    }
```

```
    for (int start = 0; start < N; start++) {  
        for (int i = 0; i < N; i++) visited[i] = 0;  
  
        path[0] = start;  
        visited[start] = 1;
```

```

    backtrack(1, start);
}

if (bestLen == 0) {
    printf("No Hamiltonian Cycle found");
} else {
    printf("Longest Hamiltonian Cycle: [");
    for (int i = 0; i < bestLen; i++) {
        printf("%d", bestPath[i]);
        if (i != bestLen - 1) printf(", ");
    }
    printf("]");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Nira is studying the N-Queens problem and needs to write a program to solve it. The program should find a valid arrangement of N queens on an N×N chessboard, ensuring that no two queens can attack each other.

Two queens attack each other if they are in the same row, the same column, or the same diagonal. Once a solution is found, display the board with queens placed correctly. A solution is guaranteed to exist for all valid values of N.

Write a program using Backtracking to solve the N-Queens problem.

Example

Input:

4 // Value of N

Output:

```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```

Explanation: The output represents a valid arrangement of queens on the 4x4 chessboard, where 1 indicates the position of a queen and 0 indicates an empty cell.

In the first row, a queen is placed in the third column. In the second row, a queen is placed in the first column. In the third row, a queen is placed in the fourth column. In the fourth row, a queen is placed in the second column. This arrangement of queens on the board ensures that no two queens threaten each other.

Input Format

The input consists of a single integer N, representing the size of the chessboard (N×N).

Output Format

Print the N×N board as N rows of N space-separated integers. Each integer is followed by a space (including the last integer on each row). Print 1 where a queen is placed and 0 for an empty cell. Print any one valid solution.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Answer

```
#include <stdio.h>
```

```

int N;
int board[25][25];

int isSafe(int row, int col) {
    int i, j;

    for (i = 0; i < col; i++)
        if (board[row][i]) return 0;

    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j]) return 0;

    for (i = row, j = col; i < N && j >= 0; i++, j--)
        if (board[i][j]) return 0;

    return 1;
}

```

```

int solve(int col) {
    if (col == N) return 1;

    for (int i = 0; i < N; i++) {
        if (isSafe(i, col)) {
            board[i][col] = 1;

            if (solve(col + 1)) return 1;

            board[i][col] = 0;
        }
    }
    return 0;
}

```

```

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    solve(0);
}

```

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        printf("%d ", board[i][j]);  
    }  
    printf("\n");  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

In chess, a knight moves in an "L" shape: two squares in one direction and one square perpendicular to it. From position (0, 0) on an infinite chessboard, a knight can move to one of the following eight positions in a single step:

(1, 2), (-1, 2), (1, -2), (-1, -2), (2, 1), (-2, 1), (2, -1), (-2, -1).

Given a target position (x, y) where both x and y are positive integers, find the minimum number of steps needed for the knight to travel from (0, 0) to (x, y) using BFS (Breadth-First Search).

Beginning from location (0,0), what is the minimum number of steps needed for a knight to get to some other arbitrary location (x,y)?

Input Format

The first line contains two space-separated integers x and y, representing the target position of the knight on the chessboard.

Output Format

Print a single integer representing the minimum number of steps needed for the knight to move from (0, 0) to (x, y).

Refer to the sample output for formatting specifications

Sample Test Case

Input: 1 2

Output: 1

Answer

```
#include <stdio.h>
```

```
#define MAX 1000
```

```
int visited[15][15];
```

```
typedef struct {  
    int x, y, d;  
} Node;
```

```
int dx[8] = {1, -1, 1, -1, 2, -2, 2, -2};  
int dy[8] = {2, 2, -2, -2, 1, 1, -1, -1};
```

```
int bfs(int tx, int ty) {  
    Node q[MAX];  
    int front = 0, rear = 0;
```

```
    q[rear++] = (Node){0, 0, 0};  
    visited[0][0] = 1;
```

```
    while (front < rear) {  
        Node cur = q[front++];
```

```
        if (cur.x == tx && cur.y == ty)  
            return cur.d;
```

```
        for (int i = 0; i < 8; i++) {  
            int nx = cur.x + dx[i];  
            int ny = cur.y + dy[i];
```

```
            if (nx >= 0 && ny >= 0 && nx <= 14 && ny <= 14 && !visited[nx][ny]) {  
                visited[nx][ny] = 1;  
                q[rear++] = (Node){nx, ny, cur.d + 1};
```

```
            }  
        }
```

```

    }
    return -1;
}

int main() {
    int x, y;
    scanf("%d %d", &x, &y);

    printf("%d", bfs(x, y));

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A customer is shopping in a supermarket and has selected N items, each with a fixed price.

The customer has a discount coupon that can be applied only if the total price of some selected items is exactly equal to a target amount S.

Write a program using Backtracking to find all distinct subsets of item prices whose sum is exactly equal to S.

Each item price can be used only once. If the input contains duplicate prices, the output must not contain repeated subsets.

Formula: A subset is valid if: $p_1 + p_2 + \dots + p_k = S$

Print each valid subset with its elements in ascending order. Print subsets in the order they are discovered by the backtracking algorithm (which processes the sorted array from left to right).

Input Format

The first line contains an integer N representing the number of items.

The second line contains N space-separated integers representing item prices. The prices may include duplicate values.

The third line contains an integer S representing the target bill amount.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element).

Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 5 10 15 20
25
Output: 5 5 15
5 20
10 15

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
int arr[25];
int n, target;
int found = 0;
```

```
void printSubset(int temp[], int len) {
    for (int i = 0; i < len; i++) {
        printf("%d ", temp[i]);
    }
    printf("\n");
}
```

```
void backtrack(int idx, int temp[], int len, int sum) {
    if (sum == target) {
```

```
    printSubset(temp, len);  
    found = 1;  
    return;  
}
```

```
if (sum > target) return;
```

```
for (int i = idx; i < n; i++) {  
    if (i > idx && arr[i] == arr[i - 1]) continue;
```

```
    temp[len] = arr[i];  
    backtrack(i + 1, temp, len + 1, sum + arr[i]);
```

```
}
```

```
int main() {  
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }
```

```
    scanf("%d", &target);
```

```
    // sort
```

```
    for (int i = 0; i < n - 1; i++) {  
        for (int j = i + 1; j < n; j++) {  
            if (arr[i] > arr[j]) {  
                int t = arr[i];  
                arr[i] = arr[j];  
                arr[j] = t;  
            }  
        }  
    }  
}
```

```
int temp[25];  
backtrack(0, temp, 0, 0);
```

```
if (!found) {  
    printf("No subset found");  
}
```

```
} return 0;
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. The problem of finding a path in a graph that visits every vertex exactly once is called?

Answer

Hamiltonian path problem

Status : Correct

Marks : 1/1

2. In the Hamiltonian Cycle problem, if a graph contains a Hamiltonian cycle, it is called a

Answer

Hamiltonian graph

Status : Correct

Marks : 1/1

3. What is the primary difficulty in finding a Hamiltonian Path in a general graph?

Answer

It is an NP-complete problem.

Status : Correct

Marks : 1/1

4. Which of the following statements is true regarding the backtracking algorithm for the Hamiltonian Cycle problem?

Answer

It explores different paths in the graph to find a Hamiltonian cycle.

Status : Correct

Marks : 1/1

5. Which algorithmic paradigm is commonly used to solve the Hamiltonian Cycle problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

6. The following paradigm can be used to find the solution to the problem in minimum time: Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K:

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

7. What is a subset sum problem?

Answer

Finding a subset of a set that has the sum of elements equal to a given number

Status : Wrong

Marks : 0/1

8. Which of the following is not true about the subset sum problem?

Answer

the dynamic programming solution has a time complexity of $O(n \log n)$

Status : Correct

Marks : 1/1

9. What is the worst case time complexity of dynamic programming solution of the subset sum problem(sum=given subset sum)?

Answer

$O(\text{sum} * n)$

Status : Correct

Marks : 1/1

10. Placing N-queens so that no two queens attack each other is called _____.

Answer

N-queen's problem

Status : Correct

Marks : 1/1

11. Where is the n-queens problem implemented?

Answer

Chess

Status : Correct

Marks : 1/1

12. Which of the following methods can be used to solve N-Queen's problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

13. Which one of the following is an application of the backtracking algorithm?

Answer

Crossword

Status : Correct

Marks : 1/1

14. For N Queens problem, the time complexity is

Answer

$O(N!)$

Status : Correct

Marks : 1/1

15. How many unique solutions are there to the 4 Queens problem, where solutions that are reflections or rotations of each other are considered the same?

Answer

2

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 21

Section 1 : Coding

1. Problem Statement

Riya, the manager of a software development company, needs to assign three developers to three different projects.

Each developer has a different cost (in terms of time and resource usage) for completing each project. Riya wants to assign exactly one developer to each project such that the total development cost is minimized.

To solve this efficiently, she decides to use the Branch and Bound technique for the Assignment Problem.

Your task is to help Riya determine the minimum total cost of assigning developers to projects.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 cost matrix.

Each cell (i, j) represents the cost of assigning the ith developer to the jth project.

Output Format

The output should display the minimum total cost of the assignment in the following exact format: "Minimum total cost: X" , Where X is the minimum total cost obtained after assigning each worker to exactly one task.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 12 19

10 5 7

15 14 13

Output: Minimum total cost: 28

Answer

```
#include <stdio.h>
```

```
#define N 3
```

```
#define INF 1000000
```

```
int cost[N][N];
```

```
int assigned[N];
```

```
int best = INF;
```

```
void backtrack(int worker, int currentCost) {
```

```
    if (currentCost >= best) return;
```

```
    if (worker == N) {
```

```
        if (currentCost < best)
```

```
            best = currentCost;
```

```
        return;
```

```
    }
```

```
    for (int job = 0; job < N; job++) {
```

```
        if (!assigned[job]) {
```

```

        assigned[job] = 1;
        backtrack(worker + 1, currentCost + cost[worker][job]);
        assigned[job] = 0;
    }
}
}

```

```

int main() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < N; i++)
        assigned[i] = 0;

    backtrack(0, 0);

    printf("Minimum total cost: %d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is the manager of a disaster relief center responsible for sending essential supplies to a flood-affected area using a single rescue truck. The truck has a limited carrying capacity, so Arjun must carefully select which supply packages to load.

Each package has a specific weight and an importance score representing how critical it is for the affected people. Arjun wants to maximize the total importance score of the packages loaded onto the truck without exceeding its weight capacity.

To solve this efficiently, Arjun uses the Branch and Bound technique, which systematically explores possible combinations of packages while pruning choices that cannot produce better results.

Your task is to help Arjun determine the maximum total importance score that can be transported.

Input Format

The first line contains an integer N, representing the number of available packages.

The next N lines each contain two integers: weight[i] value[i]

- weight[i] – weight of the package
- value[i] – importance score of the package

The last line contains an integer W, representing the maximum capacity of the rescue truck.

Output Format

The output prints a single integer representing the maximum total importance score that can be carried without exceeding the truck's weight capacity.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int w, v;
```

```
    double r;  
} Item;
```

```
Item a[1005];  
int n, W;
```

```
int cmp(const void *x, const void *y) {  
    Item *i = (Item *)x;  
    Item *j = (Item *)y;  
    if (j->r > i->r) return 1;  
    if (j->r < i->r) return -1;  
    return 0;  
}
```

```
double bound(int idx, int curW, int curV) {  
    double result = curV;  
    int totalW = curW;
```

```
    for (int i = idx; i < n; i++) {  
        if (totalW + a[i].w <= W) {  
            totalW += a[i].w;  
            result += a[i].v;  
        } else {  
            int remain = W - totalW;  
            result += a[i].r * remain;  
            break;  
        }  
    }
```

```
    return result;  
}
```

```
int best = 0;
```

```
void knap(int idx, int curW, int curV) {  
    if (curW > W) return;  
    if (curV > best) best = curV;  
    if (idx == n) return;
```

```
    double ub = bound(idx, curW, curV);  
    if (ub <= best) return;
```

```
    knap(idx + 1, curW + a[idx].w, curV + a[idx].v);
```

```

    knap(idx + 1, curW, curV);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &a[i].w, &a[i].v);
        a[i].r = (double)a[i].v / a[i].w;
    }

    scanf("%d", &W);

    qsort(a, n, sizeof(Item), cmp);

    knap(0, 0, 0);

    printf("%d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n , the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
#include <stdio.h>
```

```
#define N 10
```

```
#define INF 1000000
```

```
int n;
```

```
int cost[N][N];
```

```
int visited[N];
```

```
int bestCost = INF;
```

```
int firstMin(int i) {
```

```
    int min = INF;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i != j && cost[i][j] < min)
```

```
            min = cost[i][j];
```

```
    }
```

```
    return min;
```

```
}
```

```

int secondMin(int i) {
    int first = INF, second = INF;

    for (int j = 0; j < n; j++) {
        if (i == j) continue;

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second) {
            second = cost[i][j];
        }
    }
    return second;
}

```

```

void tsp(int level, int currCost, int currBound, int pathCost, int currNode) {
    if (level == n) {
        if (cost[currNode][0] != 0) {
            int totalCost = pathCost + cost[currNode][0];
            if (totalCost < bestCost)
                bestCost = totalCost;
        }
        return;
    }
}

```

```

for (int i = 0; i < n; i++) {
    if (cost[currNode][i] != 0 && !visited[i]) {
        int tempBound = currBound;

        pathCost += cost[currNode][i];

        if (level == 1)
            currBound -= (firstMin(currNode) + firstMin(i)) / 2;
        else
            currBound -= (secondMin(currNode) + firstMin(i)) / 2;

        if (currBound + pathCost < bestCost) {
            visited[i] = 1;
            tsp(level + 1, currCost, currBound, pathCost, i);
        }
    }
}

```

```
        pathCost -= cost[currNode][i];
    }
}
```

```
int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        visited[i] = 0;
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
}
```

```
int currBound = 0;
```

```
for (int i = 0; i < n; i++)
    currBound += (firstMin(i) + secondMin(i));
```

```
currBound = (currBound % 2 == 0) ? currBound / 2 : currBound / 2 + 1;
```

```
visited[0] = 1;
tsp(1, 0, currBound, 0, 0);
```

```
printf("Minimum Cost is: %d", bestCost);
```

```
return 0;
```

```
}
```

Status : Partially correct

Marks : 1/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 42

Section 1 : Coding

1. Problem Statement

Meera, the director of a national disaster response team, must deploy three specialized rescue units to three critical disaster sites. Each rescue unit has a different response time (in minutes) to reach each site due to distance, terrain, and accessibility constraints.

However, Meera has an additional operational rule:

Rescue Unit 1 cannot be assigned to Site 3 due to equipment limitations. Rescue Unit 2 must not be assigned to Site 1 because of route restrictions.

Meera decides to use the Branch and Bound technique to explore only valid and efficient assignments while respecting all constraints.

Your task is to determine the total response time of the best valid

deployment plan that satisfies all given restrictions.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 response time matrix.

Each cell (i, j) represents the response time (in minutes) if the ith rescue unit is deployed to the jth disaster site.

Output Format

The output should display the total response time in the following exact format: "Total Response Time: X", where X is the total response time of the best valid deployment plan.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 12 25 40
30 14 28
16 18 11

Output: Total Response Time: 37

Answer

```
#include <stdio.h>
#include <limits.h>
```

```
int cost[3][3];
int used[3];
int minCost = INT_MAX;
```

```
void solve(int row, int currCost) {
    if (row == 3) {
        if (currCost < minCost)
            minCost = currCost;
        return;
    }
```

```
    for (int col = 0; col < 3; col++) {
```

```

// constraints:
// Unit 1 (row 0) cannot go to Site 3 (col 2)
// Unit 2 (row 1) cannot go to Site 1 (col 0)

if (!used[col]) {
    if (row == 0 && col == 2) continue;
    if (row == 1 && col == 0) continue;

    used[col] = 1;
    solve(row + 1, currCost + cost[row][col]);
    used[col] = 0;
}
}
}

int main() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < 3; i++)
        used[i] = 0;

    solve(0, 0);

    printf("Total Response Time: %d", minCost);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
#include <stdio.h>
```

```
int n, W;  
int w[105], v[105];
```

```
int best = 0;
```

```

void dfs(int i, int currW, int currV) {
    if (currW > W) return;

    if (i == n) {
        if (currV > best)
            best = currV;
        return;
    }

    // take item
    dfs(i + 1, currW + w[i], currV + v[i]);

    // skip item
    dfs(i + 1, currW, currV);
}

int main() {
    scanf("%d %d", &n, &W);

    for (int i = 0; i < n; i++)
        scanf("%d", &w[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &v[i]);

    dfs(0, 0, 0);

    printf("%d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Commander Vikram is preparing a spacecraft for an interplanetary research mission. The spacecraft has a limited cargo capacity, so he must carefully decide which scientific equipment modules to load.

Each module has a scientific importance value and a specific weight. Commander Vikram wants to load the spacecraft in such a way that the total scientific value of the selected modules is maximized without exceeding the cargo weight limit.

To solve this efficiently, he uses the Branch and Bound technique applied to the 0/1 Knapsack problem, which explores possible combinations of equipment and eliminates options that cannot lead to better solutions.

Your task is to help Commander Vikram determine the maximum achievable value and the selected module weights that should be loaded into the spacecraft.

Note: The weights must be printed in reverse order of the input, meaning the algorithm starts evaluating items from the last item and prints the selected weights first.

Input Format

The first line contains an integer n , representing the number of equipment modules available.

The second line contains n space-separated integers $val[i]$, representing the scientific value of each module.

The third line contains n space-separated integers $wt[i]$, representing the weight of each module.

The fourth line contains an integer W , representing the maximum cargo capacity of the spacecraft.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, printed in reverse order of the input.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: Maximum value: 220
Selected weights: 30 20

Answer

```
#include <stdio.h>
```

```
int n, W;  
int val[15], wt[15];
```

```
int bestValue = 0;  
int bestTaken[15];  
int taken[15];
```

```
void dfs(int i, int currW, int currV) {  
    if (currW > W) return;
```

```
    if (i == n) {  
        if (currV > bestValue) {  
            bestValue = currV;  
            for (int j = 0; j < n; j++)  
                bestTaken[j] = taken[j];  
        }  
        return;  
    }
```

```
    // take item  
    taken[i] = 1;  
    dfs(i + 1, currW + wt[i], currV + val[i]);
```

```
    // not take item  
    taken[i] = 0;  
    dfs(i + 1, currW, currV);  
}
```

```
int main() {
```

```

scanf("%d", &n);

for (int i = 0; i < n; i++)
    scanf("%d", &val[i]);

for (int i = 0; i < n; i++)
    scanf("%d", &wt[i]);

scanf("%d", &W);

dfs(0, 0, 0);

printf("Maximum value: %d\n", bestValue);
printf("Selected weights: ");

// reverse order printing
for (int i = n - 1; i >= 0; i--) {
    if (bestTaken[i])
        printf("%d ", wt[i]);
}

return 0;
}

```

Status : Partially correct

Marks : 2/10

4. Problem Statement

You are tasked with solving the Travelling Salesman Problem (TSP) for a given set of cities.

Given the distances between pairs of cities, your goal is to find the shortest route that visits each city exactly once and returns to the starting city.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input contains an integer n , representing the number of cities.

The following n lines each contain n space-separated integers, representing the

distances between pairs of cities.

The distance between city i and city j is given at the i th row and j th column of the matrix.

Output Format

The first line displays "The Path is: " followed by the sequence of cities visited in the shortest path,

separated by "--->".

The second line displays "Minimum cost is X", where X is the minimum cost of the shortest path found.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1--->3--->2--->4--->1

Minimum cost is 7

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[15][15];
```

```
int bestCost = INT_MAX;
```

```
int visited[15];
```

```
int path[15], bestPath[15];
```

```
void copyPath() {
```

```
    for (int i = 0; i <= n; i++)
```

```

    bestPath[i] = path[i];
}

void tsp(int level, int currCost) {
    if (level == n) {
        if (cost[path[level - 1]][path[0]] != 0) {
            int total = currCost + cost[path[level - 1]][path[0]];

            if (total < bestCost) {
                bestCost = total;
                path[n] = path[0];
                copyPath();
            }
        }
    }
    return;
}

```

```

for (int i = 1; i < n; i++) {
    if (!visited[i] && cost[path[level - 1]][i] != 0) {
        visited[i] = 1;
        path[level] = i;

        tsp(level + 1, currCost + cost[path[level - 1]][i]);

        visited[i] = 0;
    }
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    path[0] = 0;
    visited[0] = 1;
}

```

```

tsp(1, 0);

printf("The Path is: ");
for (int i = 0; i <= n; i++) {
    printf("%d", bestPath[i] + 1);
    if (i < n) printf("---->");
}

printf("\nMinimum cost is %d", bestCost);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to location j .

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 10 8 9 7

10 0 10 5 6

8 10 0 8 9

9 5 8 0 6

7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1

Minimum cost is 34

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int n;
```

```
int cost[MAX][MAX];
```

```
int visited[MAX];
```

```
int path[MAX];
```

```
int final_path[MAX];
```

```
int final_res = INT_MAX;
```

```
int first_min(int i) {
```

```
    int min = INT_MAX;
```

```
    for (int j = 0; j < n; j++)
```

```
        if (i != j && cost[i][j] < min)
```

```
            min = cost[i][j];
```

```
    return min;
```

```
}
```

```
int second_min(int i) {
```

```
    int first = INT_MAX, second = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i == j) continue;
```

```

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second && cost[i][j] != first) {
            second = cost[i][j];
        }
    }
    return second;
}

```

```

void copy_to_final(int curr_path[]) {
    for (int i = 0; i < n; i++)
        final_path[i] = curr_path[i];
    final_path[n] = curr_path[0];
}

```

```

void tsp_rec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    if (level == n) {
        if (cost[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + cost[curr_path[level - 1]][curr_path[0]];
            if (curr_res < final_res) {
                copy_to_final(curr_path);
                final_res = curr_res;
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (cost[curr_path[level - 1]][i] != 0 && visited[i] == 0) {
        int temp = curr_bound;
        curr_weight += cost[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= (first_min(curr_path[level - 1]) + first_min(i)) / 2;
        else
            curr_bound -= (second_min(curr_path[level - 1]) + first_min(i)) / 2;

        if (curr_bound + curr_weight < final_res) {
            curr_path[level] = i;
            visited[i] = 1;

```

```

        tsp_rec(curr_bound, curr_weight, level + 1, curr_path);
    }

    curr_weight -= cost[curr_path[level - 1]][i];
    curr_bound = temp;

    for (int j = 0; j < n; j++) {
        if (visited[j]) {
            curr_path[level] = j;
            break;
        }
    }

    visited[i] = 0;
}
}

void tsp() {
    int curr_path[MAX];
    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (first_min(i) + second_min(i));

    curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = 1;
    curr_path[0] = 0;

    tsp_rec(curr_bound, 0, 1, curr_path);

    printf("Shortest Path: ");
    for (int i = 0; i <= n; i++) {
        printf("%d", final_path[i] + 1);
        if (i != n) printf(" ");
    }
    printf("\nMinimum cost is %d", final_res);
}

```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++)
```

```
            scanf("%d", &cost[i][j]);
```

```
    tsp();
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. Which of the following methods is commonly used to solve assignment problems?

Answer

Hungarian method

Status : Correct

Marks : 1/1

2. The application of assignment problems is to obtain _____

Answer

Minimum cost or maximum profit

Status : Correct

Marks : 1/1

3. The assignment problem is said to be balanced if _____.

Answer

the number of rows is equal to the number of columns

Status : Correct

Marks : 1/1

4. The minimum number of lines covering all zeros in a reduced cost matrix of order n can be _____.

Answer

at the most n

Status : Correct

Marks : 1/1

5. The assignment problem _____.

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

6. You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights of {20, 30, 40, and 70} and values {70, 80, 90, and 200}.

What is the maximum value of the items you can carry in the knapsack?

Answer

160

Status : Correct

Marks : 1/1

7. What is the objective of the knapsack problem?

Answer

To Get Maximum Total Value In The Knapsack

Status : Correct

Marks : 1/1

8. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Ignoring nodes with a weight exceeding the capacity.

Status : Correct

Marks : 1/1

9. What is the key difference between Backtracking and Branch and Bound in solving the Knapsack problem?

Answer

Backtracking guarantees an optimal solution, while Branch and Bound does not.

Status : Wrong

Marks : 0/1

10. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

11. The travelling salesman problem can be solved using _____.

Answer

a minimum spanning tree

Status : Correct

Marks : 1/1

12. In a traveling salesman problem, the elements of diagonal from left-top to right-bottom are _____.

Answer

all infinity

Status : Correct

Marks : 1/1

13. Which type of graph is used to model the TSP problem?

Answer

Undirected Weighted Graph

Status : Correct

Marks : 1/1

14. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct

Marks : 1/1

15. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5

Output: Maximum number of employees that could get works are: 6

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 50
```

```
int adj[MAX][MAX];
int matchR[MAX];
int vis[MAX];
int n;
int maxNode = 0;
```

```
int dfs(int u) {
    for (int v = 0; v <= maxNode; v++) {
        if (adj[u][v] && !vis[v]) {
            vis[v] = 1;
```

```

        if (matchR[v] == -1 || dfs(matchR[v])) {
            matchR[v] = u;
            return 1;
        }
    }
}
return 0;
}

```

```

int main() {
    scanf("%d", &n);

    memset(adj, 0, sizeof(adj));
    memset(matchR, -1, sizeof(matchR));

    int x, y;

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &x, &y);
        adj[x][y] = 1;

        if (x > maxNode) maxNode = x;
        if (y > maxNode) maxNode = y;
    }

    int result = 0;

    for (int i = 0; i <= maxNode; i++) {
        memset(vis, 0, sizeof(vis));
        if (dfs(i)) result++;
    }

    printf("Maximum number of employees that could get works are: %d", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y , representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e , representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v , representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4

4

0 1

0 2

1 3

2 3

Output: 0 3

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <limits.h>
```

```
#define MAX 105
```

```
int X, Y;  
int adj[MAX][MAX];
```

```
int pairU[MAX], pairV[MAX], dist[MAX];
```

```
int bfs() {  
    int queue[MAX], front = 0, rear = 0;
```

```
    for (int u = 0; u < X; u++) {  
        if (pairU[u] == -1) {  
            dist[u] = 0;  
            queue[rear++] = u;  
        } else {  
            dist[u] = INT_MAX;  
        }  
    }  
}
```

```
dist[-1] = INT_MAX;
```

```
while (front < rear) {  
    int u = queue[front++];
```

```
    if (dist[u] < dist[-1]) {  
        for (int v = 0; v < Y; v++) {  
            if (adj[u][v]) {  
                if (pairV[v] == -1) {  
                    dist[-1] = dist[u] + 1;  
                } else if (dist[pairV[v]] == INT_MAX) {  
                    dist[pairV[v]] = dist[u] + 1;  
                    queue[rear++] = pairV[v];  
                }  
            }  
        }  
    }  
}
```

```
return dist[-1] != INT_MAX;
```

```
}
```



```

int dfs(int u) {
    if (u != -1) {
        for (int v = 0; v < Y; v++) {
            if (adj[u][v]) {
                if (pairV[v] == -1 ||
                    (dist[pairV[v]] == dist[u] + 1 && dfs(pairV[v]))) {
                    pairU[u] = v;
                    pairV[v] = u;
                    return 1;
                }
            }
        }
        dist[u] = INT_MAX;
        return 0;
    }
    return 1;
}

```

```

void hopcroftKarp() {
    for (int i = 0; i < X; i++) pairU[i] = -1;
    for (int i = 0; i < Y; i++) pairV[i] = -1;

    while (bfs()) {
        for (int u = 0; u < X; u++) {
            if (pairU[u] == -1)
                dfs(u);
        }
    }
}

```

```

void printVertexCover() {
    int visU[MAX] = {0}, visV[MAX] = {0};
    int queue[MAX], front = 0, rear = 0;

    // free left vertices
    for (int u = 0; u < X; u++) {
        if (pairU[u] == -1) {
            queue[rear++] = u;
            visU[u] = 1;
        }
    }
}

```

```

// alternating BFS
while (front < rear) {
    int u = queue[front++];

    for (int v = 0; v < Y; v++) {
        if (adj[u][v] && !visV[v]) {
            visV[v] = 1;

            if (pairU[u] != v && pairV[v] != -1 && !visU[pairV[v]]) {
                visU[pairV[v]] = 1;
                queue[rear++] = pairV[v];
            }
        }
    }
}

```

```

// vertex cover = (Left not visited) U (Right visited)
for (int u = 0; u < X; u++) {
    if (!visU[u])
        printf("%d ", u);
}

```

```

for (int v = 0; v < Y; v++) {
    if (visV[v])
        printf("%d ", v);
}

```

```

int main() {
    scanf("%d %d", &X, &Y);

    int e;
    scanf("%d", &e);

    memset(adj, 0, sizeof(adj));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }
}

```

```
hopcroftKarp();  
printVertexCover();  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4

0 1

1 2

2 3

3 0

Output: The graph is bipartite.

Answer

```
#include <stdio.h>
```

```
#define MAX 105
```

```
int adj[MAX][MAX];
```

```
int color[MAX];
```

```
int queue[MAX];
```

```
int isBipartite(int n) {  
    for (int i = 0; i < n; i++)  
        color[i] = -1;
```

```
    for (int start = 0; start < n; start++) {  
        if (color[start] != -1) continue;
```

```
        int front = 0, rear = 0;
```

```
        queue[rear++] = start;  
        color[start] = 0;
```

```
        while (front < rear) {  
            int u = queue[front++];
```

```
            for (int v = 0; v < n; v++) {  
                if (adj[u][v]) {  
                    if (color[v] == -1) {  
                        color[v] = 1 - color[u];  
                        queue[rear++] = v;  
                    }  
                    else if (color[v] == color[u]) {  
                        return 0;  
                    }  
                }  
            }  
        }  
    }  
}
```

```
    return 1;
```

```
}
```

```
int main() {
```

```
int n, m;
scanf("%d %d", &n, &m);

for (int i = 0; i < m; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
    adj[v][u] = 1;
}

if (isBipartite(n))
    printf("The graph is bipartite.");
else
    printf("The graph is not bipartite.");
return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : Coding

1. _____ is a matching with the largest number of edges.

Answer

Maximum bipartite matching

Status : Correct

Marks : 1/1

2. How many colors are used in a bipartite graph?

Answer

2

Status : Correct

Marks : 1/1

3. What is the simplest method to prove that a graph is bipartite?

Answer

It does not have a cycle of an odd length

Status : Correct

Marks : 1/1

4. A matching that matches all the vertices of a graph is called?

Answer

Perfect matching

Status : Correct

Marks : 1/1

5. In a bipartite graph $G=(V,U,E)$, the matching of a free vertex in V to a free vertex in U called?

Answer

Augmenting

Status : Correct

Marks : 1/1

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

Answer

C

Status : Correct

Marks : 1/1

7. Which of the following is not a property of the bipartite graph?

Answer

Asymmetric spectrum

Status : Correct

Marks : 1/1

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

Answer

49

Status : Correct

Marks : 1/1

9. What does the Halting Problem attempt to determine?

Answer

Whether a program will eventually stop running or continue forever

Status : Correct

Marks : 1/1

10. The Halting Problem was introduced by which computer scientist?

Answer

Alan Turing

Status : Correct

Marks : 1/1

11. Why is the Halting Problem considered unsolvable?

Answer

Because no algorithm can determine for every program whether it halts or runs forever

Status : Correct

Marks : 1/1

12. In computation theory, what does an undecidable problem mean?

Answer

A problem that cannot be implemented in programming languages

Status : Wrong

Marks : 0/1

13. In the context of the Halting Problem, what does the term halting refer to?

Answer

A program stopping its execution after completing its task

Status : Correct

Marks : 1/1

14. What is the main implication of the Halting Problem for computer science?

Answer

Some computational problems cannot be solved by any algorithm

Status : Correct

Marks : 1/1

15. The Halting Problem generally takes which of the following as input?

Answer

A program and its input data

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 29

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

$\text{sum}(\text{subset elements}) = \text{target}$

Input Format

The first line contains an integer n , representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t , representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t , one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

3

Output: 1 2

3

Answer

```
#include <stdio.h>
```

```
int n, target;  
int arr[25];  
int subset[25];  
int found = 0;
```

```
void printSubset(int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", subset[i]);  
    }  
    printf("\n");  
}
```

```
void backtrack(int idx, int sum, int size) {  
    if (sum == target && size > 0) {  
        printSubset(size);  
        found = 1;  
        return;  
    }
```

```
    if (idx == n) return;
```

```
    // Include current element  
    subset[size] = arr[idx];  
    backtrack(idx + 1, sum + arr[idx], size + 1);
```

```
    // Exclude current element  
    backtrack(idx + 1, sum, size);  
}
```

```
int main() {  
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);
```

```
    scanf("%d", &target);
```

```
    backtrack(0, 0, 0);
```

```
    if (!found) {  
        printf("No subset matches the target sum");  
    }  
  
    return 0;  
}
```

Status : Partially correct

Marks : 9/10

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k .

Input Format

The first line contains two integers n and m , representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v , representing a connection between nodes u and v .

The last line contains an integer k , representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: $v_1 v_2 v_3 \dots$ "

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6

0 1

0 2

1 2

1 3

2 3

3 4

3

Output: Clique Found: 0 1 2

Answer

```
#include <stdio.h>
```

```
#define MAX 25
```

```
int n, m, k;  
int graph[MAX][MAX];  
int path[MAX];  
int found = 0;
```

```
int isValid(int v, int pos) {  
    for (int i = 0; i < pos; i++) {  
        if (!graph[path[i]][v])  
            return 0;  
    }  
    return 1;  
}
```

```
void printClique(int size) {  
    printf("Clique Found: ");  
    for (int i = 0; i < size; i++) {  
        printf("%d ", path[i]);  
    }  
    printf("\n");  
}
```

```
void backtrack(int start, int depth) {
```

```

    if (found) return;

    if (depth == k) {
        printClique(k);
        found = 1;
        return;
    }

    for (int v = start; v < n; v++) {
        if (isValid(v, depth)) {
            path[depth] = v;
            backtrack(v + 1, depth + 1);
        }
    }
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    scanf("%d", &k);

    backtrack(0, 0);

    if (!found) {
        printf("No clique of given size exists");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n , representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: x_1 x_2 x_3 ...
- Subset 2: y_1 y_2 y_3 ...

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 5 11 5

Output: Subset 1: 1 5 5

Subset 2: 11

Answer

```
#include <stdio.h>
```



```
#define MAX 20
```

```
int n;  
int arr[MAX];  
int subset[MAX];  
int used[MAX];  
int total = 0;  
int found = 0;
```

```
void printResult() {  
    printf("Subset 1: ");  
    for (int i = 0; i < n; i++) {  
        if (used[i])  
            printf("%d ", arr[i]);  
    }  
    printf("\nSubset 2: ");  
    for (int i = 0; i < n; i++) {  
        if (!used[i])  
            printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
void backtrack(int idx, int sum) {  
    if (found) return;  
  
    if (sum == total / 2) {  
        found = 1;  
        printResult();  
        return;  
    }  
  
    if (idx == n) return;
```

```
    // Include current element in subset 1  
    used[idx] = 1;  
    backtrack(idx + 1, sum + arr[idx]);
```

```
    // Backtrack  
    used[idx] = 0;
```

```
    // Exclude current element
```

```
    backtrack(idx + 1, sum);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        total += arr[i];
    }

    // If total is odd, no partition possible
    if (total % 2 != 0) {
        printf("No valid equal partition exists.");
        return 0;
    }

    backtrack(0, 0);

    if (!found) {
        printf("No valid equal partition exists.");
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: CHITRAPU HATHWIK GOPINATH

Email: vtu30092@veltech.edu.in

Roll no: vtu30092

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

Answer

2

Status : Correct

Marks : 1/1

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

Answer

NP

Status : Correct

Marks : 1/1

3. To which of the following classes does a CNF-satisfiability problem belong?

Answer

NP complete

Status : Correct

Marks : 1/1

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

Answer

X is in NP, but not necessarily NP-complete

Status : Correct

Marks : 1/1

5. A problem in NP is NP-complete if

Answer

It can be reduced to any other problem in NP in polynomial time

Status : Wrong

Marks : 0/1

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

Answer

P

Status : Correct

Marks : 1/1

7. What is the key property of problems in NP?

Answer

Their solutions can be verified in polynomial time

Status : Correct

Marks : 1/1

8. What does it mean if a problem is in both NP and NP-Complete?

Answer

It is the hardest among NP problems

Status : Correct

Marks : 1/1

9. If $P = NP$ is ever proven, what would it imply?

Answer

All problems in NP can be solved in polynomial time

Status : Correct

Marks : 1/1

10. What is the class NP named after?

Answer

Nondeterministic Polynomial time

Status : Correct

Marks : 1/1

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

Answer

NP-hard

Status : Correct

Marks : 1/1

12. What is the defining property of NP-hard problems?

Answer

They are at least as hard as the hardest problems in NP.

Status : Correct

Marks : 1/1

13. In computational complexity theory, what does "NP" stand for?

Answer

Non-Deterministic Polynomial Time

Status : Correct

Marks : 1/1

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

Answer

NP-complete problems are the hardest problems in NP.

Status : Correct

Marks : 1/1

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

Answer

NP-hardness

Status : Correct

Marks : 1/1